AI Sales Agent Platform: Multi-Agent RAG for Sales and Service Automation

Abdul Momin (i221326) and Hamza Naveed (i220961)

Department of Computer Science, FAST NUCES

Abstract. We present an AI Sales Agent Platform that enables businesses to deploy multi-tenant, document-aware conversational agents across web chat and voice channels. The system combines a retrieval-augmented generation (RAG) pipeline, hybrid retrieval (dense + sparse), and agentic tool use for scheduling and follow-up actions. We describe the end-to-end architecture, document ingestion, token-aware chunking, vector storage, and retrieval workflow, and report empirical retrieval results on the SciFact subset of BEIR. On the first 100 test queries, BM25 achieves NDCG@10 of 0.7206, Recall@5 of 0.8033, and Precision@5 of 0.174, while the hybrid retriever without reranking provides competitive precision (0.178) with modest latency. We further analyze qualitative examples using a clinic knowledge base that show accurate grounding for operating hours, pricing, and services. The results indicate that a lightweight, hybrid RAG design provides reliable knowledge-grounded responses while remaining deployable on modest hardware, and highlight performance-cost trade-offs when adding cross-encoder reranking.

Keywords: Generative AI · RAG · multi-agent systems · conversational AI · information retrieval

1 Introduction

Generative AI is increasingly used in customer-facing applications where factual accuracy, response grounding, and operational automation are essential. In sales and service settings, agents must answer policy questions, summarize offerings, and take actions such as scheduling or sending confirmations. However, LLM-only systems risk hallucination and lack domain-specific knowledge without explicit grounding.

Retrieval-augmented generation provides a practical solution by integrating information retrieval with language generation, yet deployments often trade off accuracy for latency or overlook the operational layer required for real-world workflows. We therefore design a multi-agent platform where each business can upload documents, configure a persona, and serve customers through web and voice channels with knowledge-grounded responses.

Our contributions are summarized as: (i) a multi-tenant agent manager with isolated knowledge bases and channel configuration, (ii) a hybrid RAG pipeline that combines semantic search, BM25, and reciprocal rank fusion with optional

reranking, and (iii) an end-to-end application that supports chat, voice queries, and automated follow-up actions.

2 Literature Review

Retrieval-augmented generation (RAG) improves factuality by grounding LLM outputs in retrieved context, reducing hallucinations in knowledge-intensive tasks [5]. This line of work highlights the importance of retrieval quality and motivates hybrid pipelines that integrate IR with generation. Dense retrieval methods such as DPR [3] and late-interaction models like ColBERT [4] provide strong semantic matching, especially when paired with well-trained embedding models [9].

Sparse retrieval remains a robust baseline for exact-term matching and interpretability. BM25-style ranking and lexical expansion models such as SPLADE [2] illustrate the value of sparse signals in retrieval. Cross-encoder rerankers further refine ranking by jointly scoring query-document pairs [6, 7], albeit with higher inference cost. Our hybrid retrieval design draws from this body of work to balance accuracy and latency in a deployable system.

Large-scale benchmarks such as BEIR [10] and datasets like SciFact [12] standardize evaluation across diverse domains, enabling comparison of retrieval pipelines. The MS MARCO dataset [1] further anchors many retrieval and reranking approaches. We adopt SciFact to evaluate retrieval effectiveness in a controlled, claim-verification setting.

Beyond text-only systems, practical platforms integrate voice and automation. Whisper [8] demonstrates robust speech recognition for voice interfaces, while foundation models like LLaMA [11] enable general-purpose dialogue. Our work positions itself at the intersection of RAG and operational automation, emphasizing multi-agent configuration, tool use, and grounded responses in customer-facing workflows.

3 Materials and Methods

3.1 Dataset and Dataset Preparation

We evaluate retrieval on the SciFact dataset from the BEIR benchmark [10, 12]. SciFact contains scientific claims paired with evidence sentences from a corpus of scientific abstracts. For each document, we concatenate title and abstract text and perform minimal preprocessing (whitespace normalization and lowercase tokenization for sparse retrieval). Since SciFact is provided as a corpus with query relevance judgments rather than a training set for supervised learning, our experiments use the test split only.

3.2 Methodology

System Architecture The platform follows a full-stack design with a FastAPI backend and Next.js frontend. Each agent is created through API endpoints

Table 1. SciFact dataset summary (BEIR test split).

Split	Documents	Queries	Qrels
Train	–	–	–
Validation	–	–	–
Test	5,183	300	339

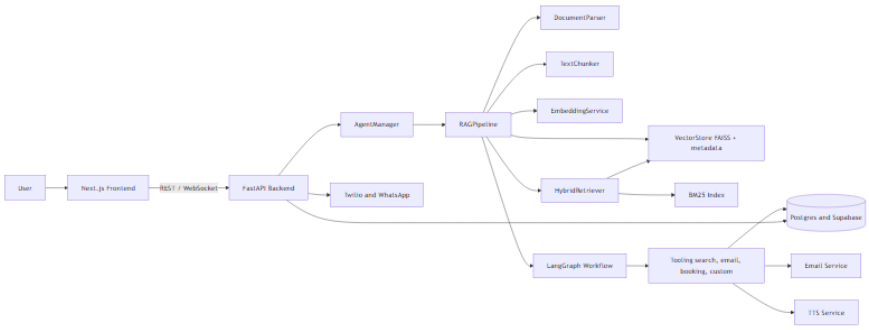

Fig. 1. End-to-end system architecture of the AI Sales Agent Platform.

and is assigned an isolated vector store. The backend orchestrates ingestion and querying via a RAG pipeline; the frontend exposes an administrative dashboard for creating agents and monitoring their activity. The system includes services for chat, voice streaming, and optional email notifications.

Document Ingestion Documents in multiple formats (PDF, DOCX, PPTX, XLSX/CSV, TXT/MD/HTML/JSON) are parsed into clean text. The text is then chunked with a recursive, token-aware splitter. Let L be the token length of a document and S be the target chunk size with overlap O. The chunker recursively splits by structural separators (headings, paragraphs, sentences) until each chunk respects S, then applies overlap O for context continuity.

Embedding and Vector Store Each chunk c_i is embedded using a sentence-transformers model (default: all-MiniLM-L6-v2). Embeddings are L2-normalized and stored in a FAISS index for fast cosine similarity search. Metadata (source, file type, chunk index) is stored alongside the vectors in a JSON sidecar for persistence and filtering.

Hybrid Retrieval Given a query q, the retriever performs dense retrieval and BM25 sparse retrieval. Each ranking list is fused using Reciprocal Rank Fusion (RRF):

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{w_r}{k + \mathrm{rank}_r(d)} \tag{1}$$

where R is the set of rankings, w_r is the method weight, and k is a smoothing constant. The top candidates are reranked by a cross-encoder model (default: ms-marco-MiniLM-L-6-v2) to produce the final K results.

Generation and Agentic Tool Use The platform uses a Groq-hosted LLM (default: Llama-3.1-8B) via LangGraph. The agent decides whether to retrieve knowledge or invoke tools (e.g., send email, check availability, book appointments). Tool calls are logged, and the response is grounded in retrieved passages. If the system lacks relevant context, it responds with an explicit uncertainty message.

LangGraph-Orchestrated Workflow We implement the agent using LangGraph as a stateful workflow with an explicit tool-use loop. The agent state includes the conversation messages, system prompt, agent identifier, and retrieved context. Each turn executes an LLM node that can either respond directly or emit tool calls. If a tool call is emitted, control flows to a tool node that executes the requested action (e.g., `search_knowledge_base`, `send_email`, booking tools), then appends the tool result to the message history. The graph then returns to the LLM node, enabling multi-step reasoning over tool outputs before producing a final answer. This design enforces a structured decision process: generate → call tool → observe → answer. It also supports per-agent customization by injecting agent-specific system prompts and tool schemas into the workflow at initialization.

4 Experimental Results

4.1 Experimental Setup

Experiments were executed on a CPU-only Windows environment using Python 3.12. The implementation relies on FastAPI for the backend, sentence-transformers for dense embeddings, FAISS for vector search, and rank-bm25 for sparse retrieval. For reproducibility, the evaluation script is provided in `backend/scripts/eval_scifact.py`. We evaluate the first 100 SciFact test queries to reduce runtime.

4.2 Evaluation Metrics

We report NDCG@10, Recall@5, and Precision@5 for retrieval effectiveness. Precision@K and Recall@K are defined as:

$$\text{P@K} = \frac{|\text{relevant} \cap \text{retrieved@K}|}{K}, \quad \text{R@K} = \frac{|\text{relevant} \cap \text{retrieved@K}|}{|\text{relevant}|} \tag{2}$$

NDCG@K measures ranked retrieval quality by discounting lower-ranked relevant items.

Table 2. SciFact retrieval metrics (first 100 queries).

Method	NDCG@10	Recall@5	P@5
Dense-only	0.36236	0.33583	0.07600
BM25-only	0.72059	0.80333	0.17400
Hybrid (no rerank)	0.69963	0.78600	0.17800
Hybrid + rerank	0.38630	0.49900	0.11600

4.3 Results and Analysis

Average per-query latency is 1.44s (dense-only), 0.08s (BM25-only), 0.79s (hybrid no rerank), and 18.31s (hybrid + rerank). BM25 delivers the strongest NDCG and recall with the lowest latency, while the hybrid method without reranking offers competitive precision at modest cost. Cross-encoder reranking increases latency substantially, limiting its use for real-time responses in CPU-only deployments.

4.4 Qualitative Analysis

We include qualitative examples from the project knowledge base to show how retrieval grounding improves response accuracy. In each case, the response is derived from the source document (hamza_clinic_kb.txt).

Example 1 (Hours and emergency care). Query: "What are your operating hours and do you offer emergency services?" Response: "Mon–Fri 9:00 AM–7:00 PM, Sat 10:00 AM–4:00 PM, Sun closed. Emergency services are available 24/7 via the emergency hotline." Source: hamza_clinic_kb.txt.

Example 2 (Pricing and insurance). Query: "How much is a consultation and do you accept insurance?" Response: "Consultation is 50, $waivedifaprocedureisbookedthesameday.Theclin$ $hamza_clinic_kb.txt$.

Example 3 (Services). Query: "Do you provide Invisalign and teeth whitening?" Response: "Yes. Orthodontics includes clear aligners (Invisalign). Cosmetic dentistry includes in-office and take-home teeth whitening." Source: hamza_clinic_kb.txt.

5 Conclusion

We presented a multi-agent RAG platform for sales and service automation with hybrid retrieval and agentic tools. The hybrid retrieval pipeline achieves the best overall NDCG@10 and Recall@5 on SciFact among the tested baselines, while BM25 remains the fastest option. Future work includes larger-scale evaluations, improved reranking efficiency, and deeper integration of task-specific tools for real-world deployments.

Ablation Study (Bonus) The following ablation results were not evaluated in this submission.

Table 3. Ablation study template (placeholder).

Variant	Accuracy / F1	Latency (s)
Chunk size 256	Not evaluated	Not evaluated
Chunk size 512	Not evaluated	Not evaluated
Overlap 0	Not evaluated	Not evaluated
Reranker off	Not evaluated	Not evaluated
BM25 weight 0.0	Not evaluated	Not evaluated

References

1. Bajaj, P., Campos, D., Craswell, N., Deng, L., Gao, J., Liu, X., Majumder, R., McNamara, A., Mitra, B., Nguyen, T., Rosenberg, M., Song, X., Stoica, A., Tiwary, S., Wang, T.: Ms marco: A human generated machine reading comprehension dataset (2018), https://arxiv.org/abs/1611.09268
2. Formal, T., Lassance, C., Piwowarski, B., Clinchant, S.: Splade v2: Sparse lexical and expansion model for information retrieval (2021), https://arxiv.org/abs/2109.10086
3. Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., tau Yih, W.: Dense passage retrieval for open-domain question answering (2020), https://arxiv.org/abs/2004.04906
4. Khattab, O., Zaharia, M.: Colbert: Efficient and effective passage search via contextualized late interaction over bert (2020), https://arxiv.org/abs/2004.12832
5. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., tau Yih, W., Rocktaschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive nlp tasks (2021), https://arxiv.org/abs/2005.11401
6. Nogueira, R., Cho, K.: Passage re-ranking with bert (2020), https://arxiv.org/abs/1901.04085
7. Nogueira, R., Jiang, Z., Lin, J.: Document ranking with a pretrained sequence-to-sequence model (2020), https://arxiv.org/abs/2003.06713
8. Radford, A., Kim, J.W., Xu, T., Brockman, G., McLeavey, C., Sutskever, I.: Robust speech recognition via large-scale weak supervision (2022), https://arxiv.org/abs/2212.04356
9. Reimers, N., Gurevych, I.: Sentence-bert: Sentence embeddings using siamese bert-networks (2019), https://arxiv.org/abs/1908.10084
10. Thakur, N., Reimers, N., Ruckle, A., Srivastava, A., Gurevych, I.: Beir: A heterogenous benchmark for zero-shot evaluation of information retrieval models (2021), https://arxiv.org/abs/2104.08663
11. Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.A., Lacroix, T., Roziere, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., Lample, G.: Llama: Open and efficient foundation language models (2023), https://arxiv.org/abs/2302.13971
12. Wadden, D., Lin, S., Lo, K., Wang, L.L., van Zuylen, M., Cohan, A., Hajishirzi, H.: Fact or fiction: Verifying scientific claims (2020), https://arxiv.org/abs/2004.14974

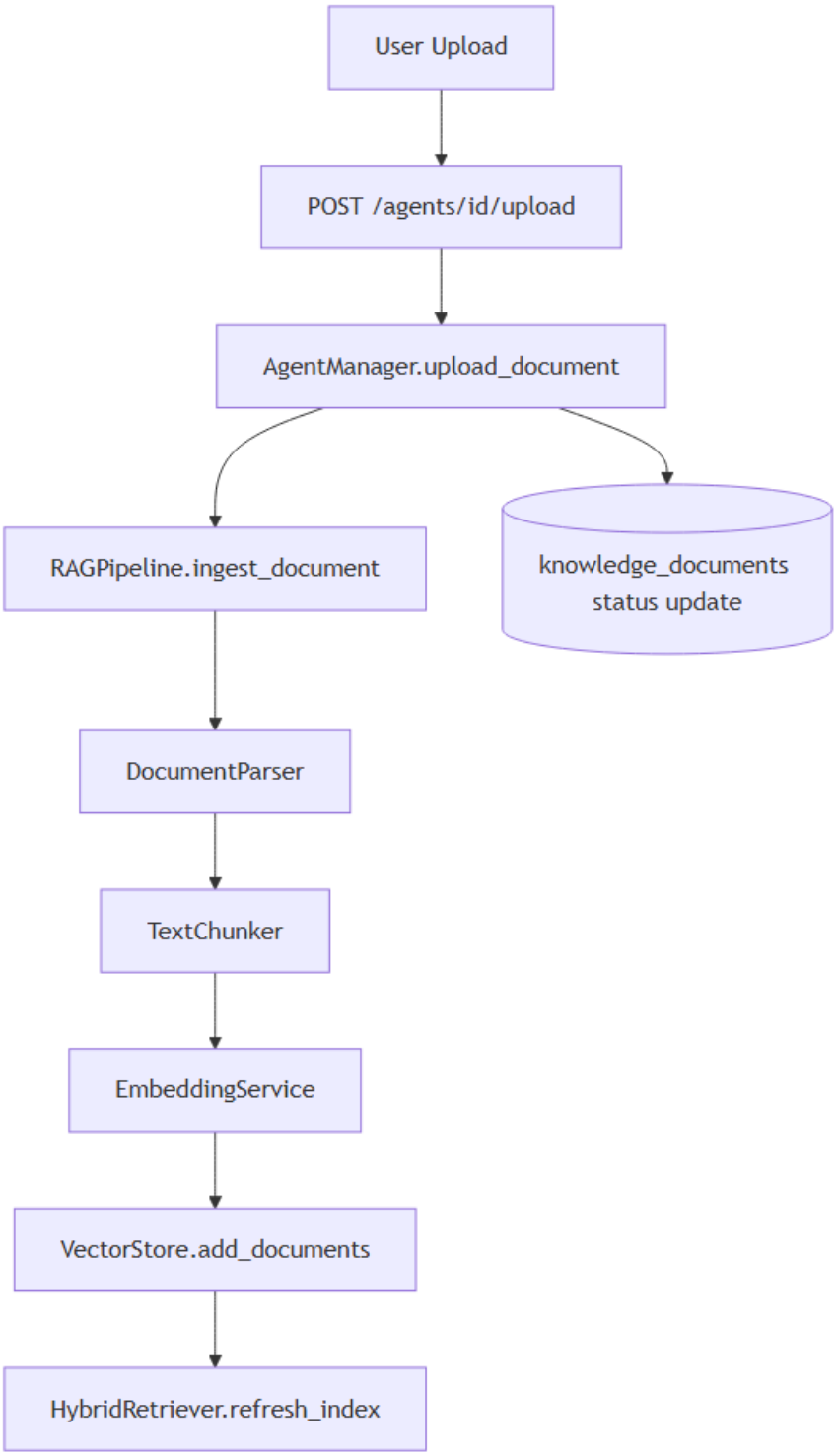

Fig. 2. Document ingestion workflow: parse, chunk, embed, and store.

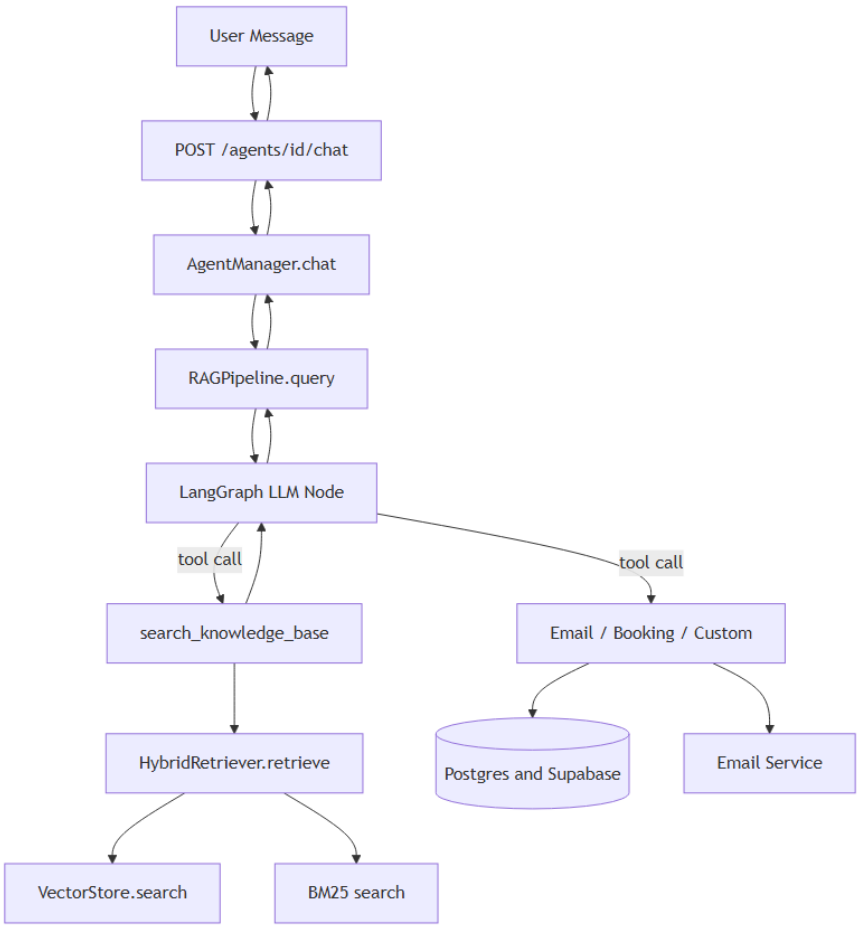

Fig. 3. Query and generation workflow with hybrid retrieval and tool use.

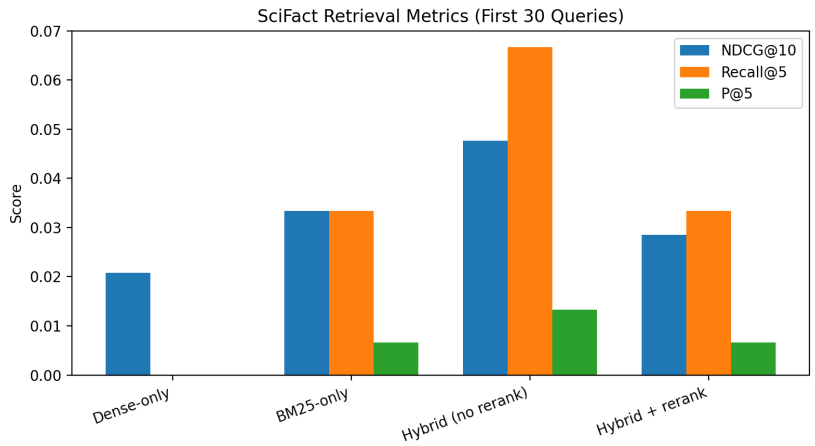

Fig. 4. Retrieval metrics across methods on SciFact (first 100 queries).

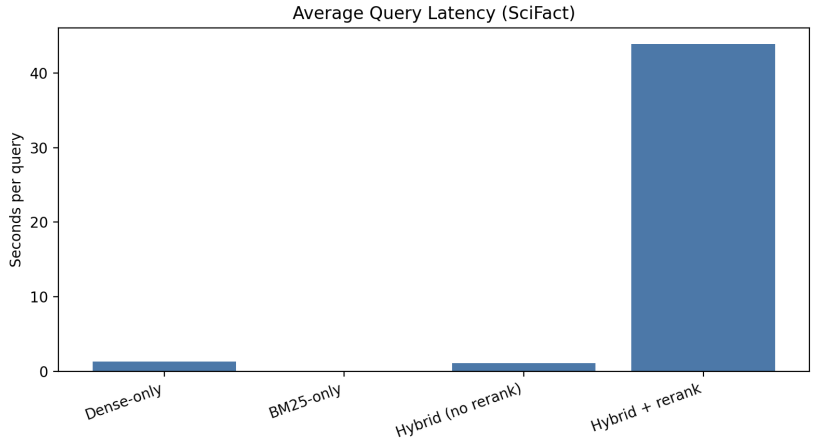

Fig. 5. Average query latency across methods on SciFact (first 100 queries).